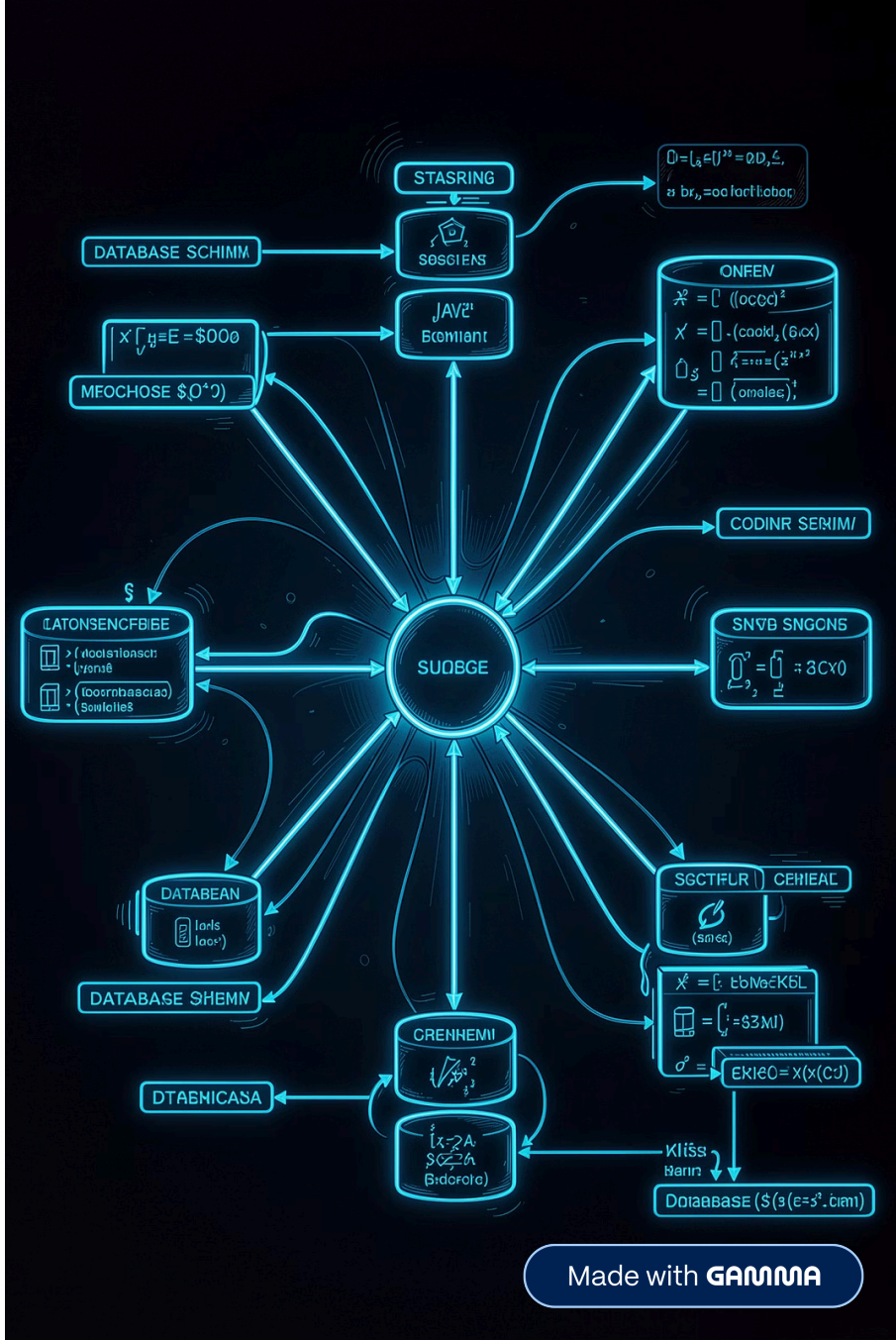
Database Design for Backend Engineers

Practical, industry-focused data modeling for security-minded developers.

Design entities first. Tables come later.

ENGINEERS.STYLE

BACKEND ENGINEERING



Database Design Is About Modeling Reality

A database should represent real-world business objects — not just rows and columns.

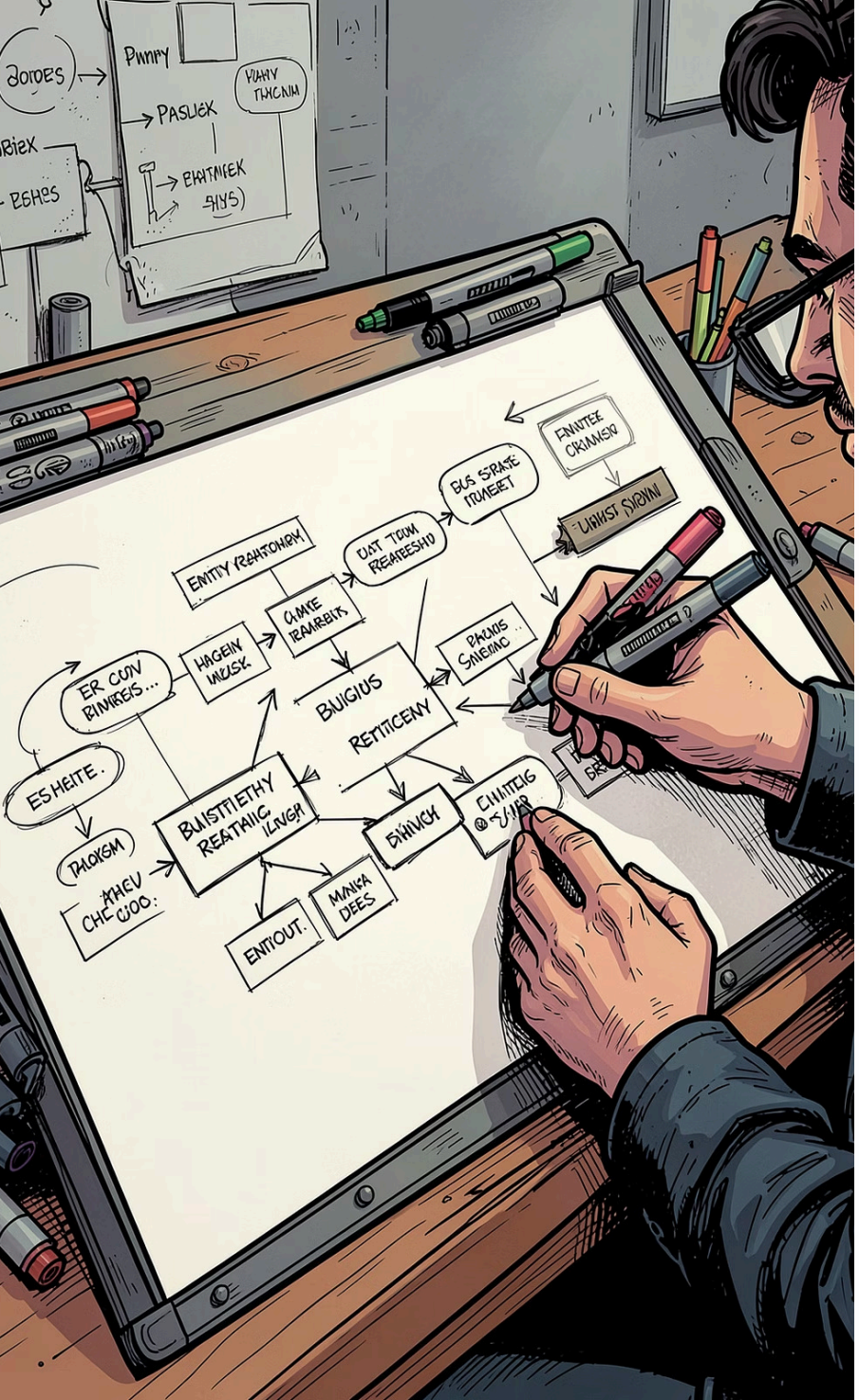
E-Commerce Example

- User
- Product
- Order

Mini ASM Example

- Asset
- Finding
- Scan
- User

Rule: Design entities first. Tables come later.



Identify Core Entities

Start by asking: **What are we managing? What information do we need? How are entities related?**

Asset

The core object being tracked — domains, IPs, services

Finding

Security issues discovered during scans

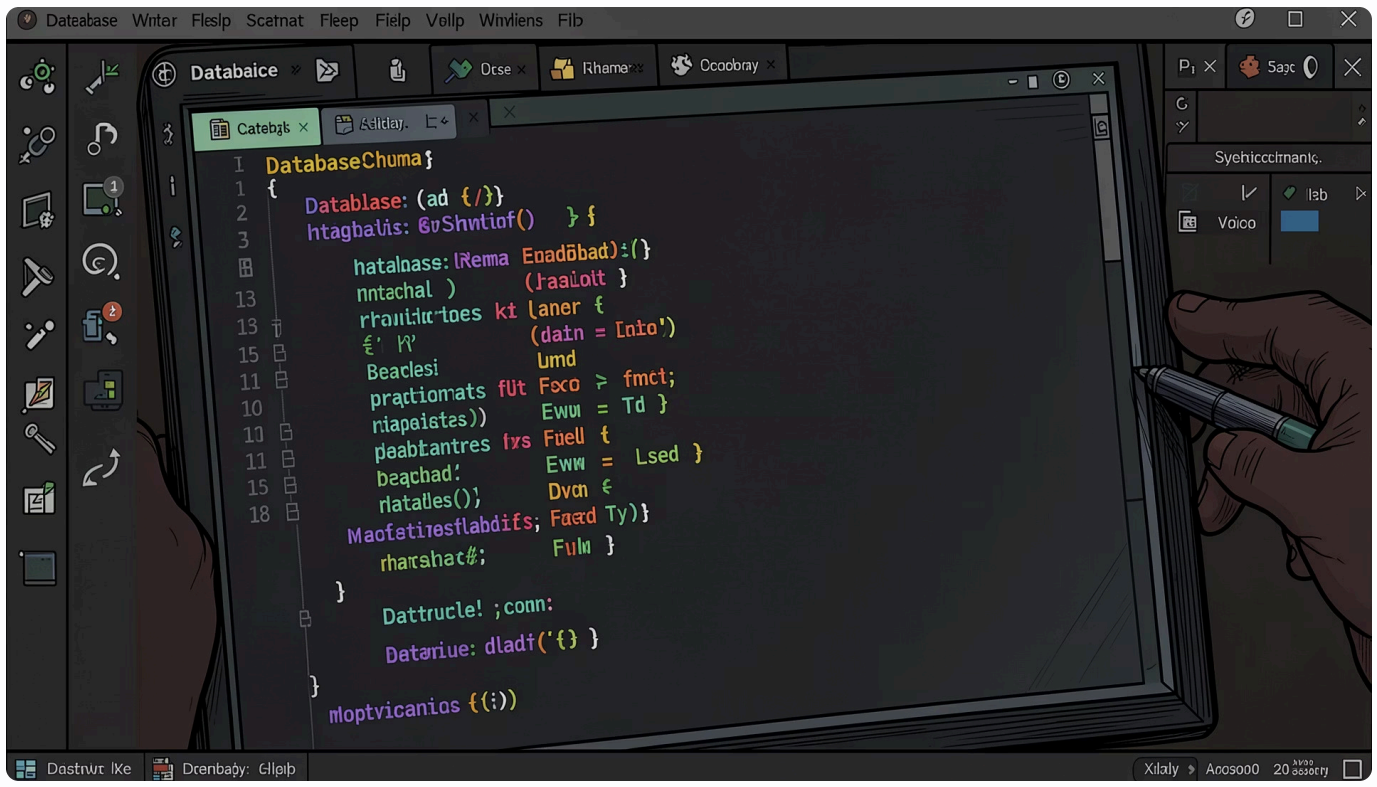
Scan

Record of each scan operation performed

User

People interacting with the system

Design Fields Carefully



Asset Entity — Example Fields

id

Unique identifier

name

Human-readable label

type

domain, ip, or service

status

active, inactive, archived

created_at

Timestamp for auditing

⚠ **Rule:** Ask "Why does this field exist?" for every column. If you can't explain it, don't add it.

Choosing a Primary Key

Your primary key is the identity of a record. It must never change.

❌ Bad Choices

asset_name

Names can be updated

domain_name

Domains can be reassigned

email

Users change email addresses

✅ Better Choices

- **UUID** — Globally unique, no coordination needed
- **Auto-Increment ID** — Simple, compact, fast

Primary keys must be: **Unique · Stable · Never changed**

Required Fields vs. Optional Fields

Required — Always Present

- **id** — Primary key, always required
- **name** — Core identity of the asset
- **type** — Determines how the asset is handled
- **created_at** — Audit trail requirement

Optional — When It Makes Sense

- **description** — Not every asset needs one
- **owner** — May be unassigned initially

Rule: Make fields required unless there's a good reason not to. Defaults reduce null-handling bugs.

Use Constraints to Protect Data

Never trust application validation alone. Enforce rules at the database level.

NOT NULL

Guarantees a field always has a value

UNIQUE

Prevents duplicate entries on key fields

CHECK

Restricts values to an allowed set

Example: `CHECK (type IN ('domain', 'ip', 'service'))` — ensures type is always valid at the database level.

One Table, One Responsibility

Avoid "god tables" that mix multiple concerns. Separate responsibilities into dedicated tables.

❌ Bad — God Table

```
assets
├── asset_name
├── asset_owner
├── owner_email
└── department_name
```

✅ Good — Separate Tables

assets

Asset data only

users

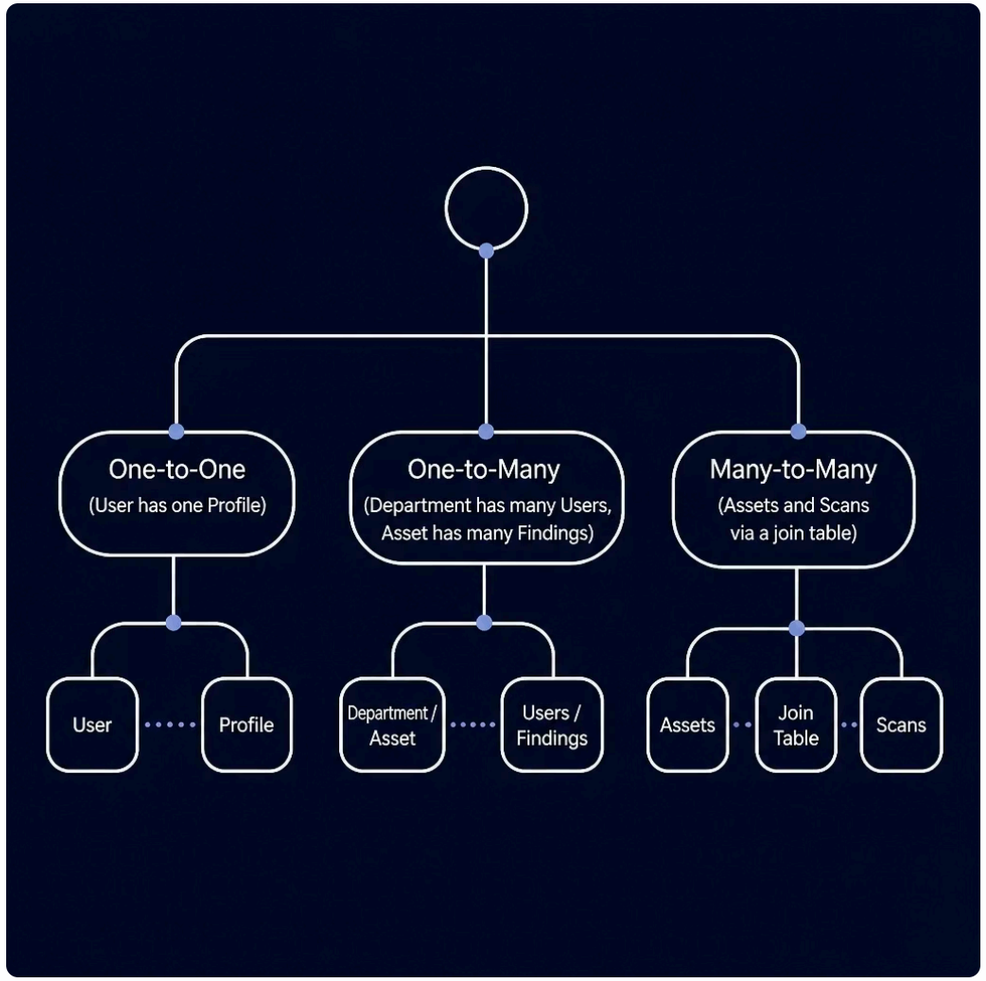
User info + email

departments

Department records

Link them with foreign keys — not duplicated columns.

Relationships Between Tables



Relationships Should Reflect Reality

Model relationships explicitly with foreign keys — not implied through naming conventions.

→ One-to-One

User ↔️ Profile — one record each

→ One-to-Many

Department → Users, Asset → Findings

→ Many-to-Many

Assets ↔️ Scans via a join table

Database Design Checklist

Before creating any table, run through this list.

01

What business entity is this?

02

Is the primary key stable?

03

Which fields are required?

04

What constraints are needed?

05

Are relationships clear?

06

Will this scale to 1M+ rows?

Key Takeaway: Good database design reduces future problems. Normalize for correctness. Denormalize only when performance requires it.